Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра «Информационная безопасность»

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №3

по дисциплине
Информационная безопасность и защита информации

Шифр Виженера
Вариант 13

Преподаватель _____ А. К. Зырянова
подпись, дата — инициалы, фамилия

Студент КИ24-07Б, 03254438 _____ А. Д. Шахов
номер группы, зачетной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026

1. Задания к практической работе

Используя методы криптоанализа шифров полиалфавитной подстановки расшифруйте текст сообщения (согласно вашему персональному варианту), а также определите секретный ключ, которым оно было зашифровано.

Известные сведения о шифрограмме следующие:

- для зашифрования сообщения, которое представляет собой отрывок из литературного произведения, использовался шифр Виженера, с алфавитом, представленным в таблице;
- знаки препинания, а также знак пробела не входят в алфавит шифрования и, соответственно, не является частью шифрограммы, т.е. шифрограмма с расставленными знаками препинания и пробелами эквивалента шифрограмме без них;
- весь шифротекст представляет собой одну строку;
- в качестве ключа шифрования использовалось одно слово (из букв алфавита), длина которого не более 10 символов.

Для осуществления статистического криптоанализа необходимо разработать программу дешифрации со следующими возможностями:

- автоматическое приближённое нахождение периода шифрования с помощью теста Касиски;
- автоматическая проверка правильности выбора числа m с помощью нахождения индексов соответствия (для всех подгрупп шифротекста);
- в случаях, когда тест Касиски не удаётся применить или значение числа m, найденное с его помощью, не подтверждается после определения индексов соответствия, m определяется подбором (такой случай возможен в случаях, когда длина ключа более 7 знаков и/или зашифрованный текст слишком короткий);
- автоматическое нахождение индексов взаимного соответствия для m зашифрованных столбцов;

- автоматическое вычисление всех возможных вариантов ключа шифрования, и вывод их пользователю для принятия решения;
- расшифровка шифротекста, по выбранному пользователем ключу шифрования.

Персональный вариант представлен ниже:

Вариант 13.

ЧЩ ЭГТБОР ЫШУАХУ ЪО ЮЕКСУЯ, ЧНШ ЭОЭЫЭ ЛЖЩ ВЫЛЭВЦЭ ЬГЪЩСБП, ИП ЧУЛНАЬ ХЬШ-ДНШ ЫОДЮЕКТОЕЙ Й ЬЩЪ БОФПЬАХТ, Е ФЩАБЭЙЦЮ ЭГХИКПЩШУЫХ ЪЦГЫЮШН. УДЯЧ ШЯЦЙТЛЕ, ЬЬДЯЫМЦНСЖГП Н ЭШЯАЪМБГРА ФЛШБЦ-НШ ЬЮШСИТФ ШЮНМЫ ЩПМТМЬНО. АН ЭПВУГТ, ИК ЩПШСА Э ЬАУЯМФЩСБ ЮЙМРААХЕК УЩЫ А ЯПФЯЕПГЪРЩПЪЙНЩ ЯЕННЫХЬЦЫ, ЕШЭЬГИД МЖЯЮАБТЦ ЛЕЫН БУЫ ДЫЛШХОЮТНЧУЪЫ ЯЛЭПОЯХ, ЭЕ НЯШРЯК ШОЬСАЬР ЬЧЪЙНЩ ЦЪ ЪГЯ. ШУДЧЙХЗШБ ОЖППЫОВ, МШНУГЕАЧШЬ ФТМАНУЕЪЦЯ, ХОЭ ЬАЬРЮФАЛН, ПЬКТЛПФ, ЦЪ ЧЙЬЩЮОВ ГЧЖУ ВТЛПТЮШШГ, БЛЧАИД ЫЭЬЮХЕ, ЮЩЮЕТКТЛЫБ, СЙЦЛЖАХА ЬЛЫЙИ - ЭЫР ЛЕЫ ЬЕНОШЯ ИПЫОЪСАХЗЫБ Ю ММРАЮИЗ ЗЪЬЮТНШЧ, ШБЯЙЪЖЧ ФШАГРА ВЭГ ХЛЪВТ, ЗПСТЖ ОЖКОЬАЭЫМШЬЬ ОЖШШТЫЪЕШФ Ц КТЛЧЖЪ ЗЭЫФЩЪ ФЭЫЬБО ЫШГ ОЩЪУЕИПОЬ ЪЪЫФЩЪБРЙ. ЗЭЦИ ВЖКПЫБЧЛШНЫОВ ЯПНЦЙ ДЛПТРОДЫУШЬ ЕЭООШЬ ГНМВРРШШГЪЗ Ц ЪНМЬЛРЫЯЧ ЫЧУТЯЧЫК; ТЮМ ШЬЩСБ ЪОРШЬ ФЫЖЖГЫШ ХМФЮЯДЯЭШ УЩЫ, ШОБГУ ДЧЫСЛАП, ЮЙМЬУЯ ЪА ТЧУЕЙ ИТХОЭЫЮШ УЯЭАМЫЭРУ. ЪОРШЬ ЦЫЭШЫЦЕЙ НКХ, ЕЕЫЬЕ ШУ ФИЖШ ШЦ ДШГВХЬЯ АЗЧЩ, ЫЫ ЮЖТГШБЩ МЦРЖАЫ, ТЬЩПО ПЙ МЬУЯ ОЦХЛ АУ ЩАХЩЕП, ЧЙЬЩЮЖЛ ЖИМНЕ УАЧДЦАИ. Э ЗЭЬЯ ЪЫОЩПАЫ ЙЬПОЕЙ МЩЫОХТЯХУРБЮНЖ ЩХАНТПШЫОЩ ЮШЬЭБСЫЦ. ЩЫЫ ХЗПЙА БЮЙЛРЫАИД ОЛЮ ЪНМЬЛРЮМНЖ ЬЪШМНЖЬН Ы ЮЖЭГОЕЙ ШЬУГ ФТМАНУЕЪЦЯ ХЮУЮЫМУД. ХЫМФЦЦЙНИТК, ХУСОВЛУЯИА ЫЧУИЫЗ: "КА, ЭШЭАЫЭОАЙНП! ШУ

ДЯЦОШЬ ЮХ ЭКЧ АУЧ МЦРЖЫЯЧ!" - ЛЖРУЛН ТЧ ЕУЮНШ ЦБКЕАИ ШОЦЭЫОЩМ. Х ПЦЫГУЯ ЧЖКЬЯШ ЫИТ ЪЬВНЯКЙАДМ ЙБРЫП ЭАОХЬ ЫШГ, ХЮЕЛТ МФЛХУЯЧ, ЧУШБРЯК. ЩАЕАЯК ЩЫЫ ЮЙМРЮЛТИЧЩ РОЯАЫШУАИ НПЧ, ЕЕЫ ИКТЙХНЩЬ Н ЛЕЫЗ ШМЗШЮНМР ОГХМЬЩШГННКЧЦ; ХЬЛШВУЯ, ЫИТ ЬЕЫЯЫИЭЯТ АТПШЙЯХ Г МЩЯВХНКШЫОЩГ ХЙТПЩГ. ШШЦ ЮЛЬЙЭ ЭБЯЙХХЬХННЖ ЩП ЮХНПЫОЕАЛП; АРУШЪЬ МБЮРЫЪУЫУ, ЬОВХЦАН Г НЫУКН Г НЩРБЭЪЬ Ь ЭГТВЪРЫЫТЗ Т ЩЯЕЭЙЭЧЫОЩГ ФЩЩЭЫМЬКЪЫ ЫЬ К. Л. ЬГШЙМР.

2. Описание разработанной программы

2.1. Назначение программы

Программа предназначена для автоматического взлома шифра Виженера без знания ключа. Она реализует полный цикл статистического криптоанализа: от определения длины ключа до подбора конкретного слова и финальной расшифровки сообщения.

2.2. Используемые алгоритмы

Тест Касиски: Поиск повторяющихся последовательностей символов (триграмм) для выявления периодов, кратных длине ключа.

Индекс совпадений (Index of Coincidence, IC): Статистическая мера, позволяющая отличить случайный набор букв от текста на естественном языке. Используется для подтверждения длины ключа и оценки качества подбора сдвигов.

Частотный анализ: Сравнение распределения букв в колонках текста с эталонными частотами русского языка для определения букв ключа.

Метод циклических сдвигов: Генерация всех возможных ротаций

найденного базового ключа для устранения неоднозначности начала слова.

2.3. Руководство пользователя

Запуск: Запустите файл pr3-v13.py. Откроется графическое окно приложения.

Ввод данных: В верхнее текстовое поле «Введите текст» вставьте или наберите шифрограмму. Программа автоматически принимает текст с пробелами и знаками препинания.

Анализ:

Нажмите кнопку «Анализировать».

Программа очистит текст от шумовых символов.

Выполнит тест Касиски и расчет индексов совпадений.

Определит наиболее вероятную длину ключа.

Сгенерирует список возможных ключей (циклические сдвиги).

Выбор ключа:

В списке «Выберите ключ» отображаются варианты ключей.

Просмотрите список.

Кликните левой кнопкой мыши по строке с ключом.

Получение результата:

В нижнем поле «Расшифрованный текст» автоматически появится полное расшифрованное сообщение с сохранением всех пробелов, знаков препинания и абзацев исходного текста.

3. Поэтапное описание дешифрации

Этап 1: Предобработка данных

Исходный текст был загружен в программу. Все символы, не входящие в алфавит (пробелы, знаки препинания, переносы строк), были временно удалены

для проведения статистического анализа. Текст приведен к верхнему регистру, буква «Ё» заменена на «Е».

Этап 2: Определение длины ключа

Тест Касиски: Программа обнаружила повторяющиеся триграммы. Были вычислены расстояния между их вхождениями. Анализ делителей этих расстояний показал наибольшую частоту для чисел: [перечисли топ-3 кандидата, например: 2, 3, 6].

Проверка через IC: Для кандидатов была рассчитана средняя величина индекса совпадений по подгруппам.

Для длины 2: IC = ...

Для длины 3: IC = ...

Для длины 6: IC = 0.058 (наиболее близко к эталонному 0.055 для русского языка).

Результат: Длина ключа утверждена равной

m = 6.

Этап 3: Восстановление ключа

Текст был разбит на 6 колонок (подгрупп).

Для каждой колонки методом частотного анализа (сравнение с эталоном) была найдена наиболее вероятная буква сдвига.

Сформирован базовый ключ: ЛАПОТЬ.

Так как статистический метод может давать циклический сдвиг, программа сгенерировала все 6 вариантов ротации этого ключа:

КЛОУНЫ

ЛОУНЫК

ОУНЫКЛ

УНЫКЛО

НЫКЛОУ

ЫКЛОНУ

Этап 4: Выбор правильного варианта и расшифровка

Пользователь визуально проанализировал превью расшифровки для

каждого варианта.

При выборе ключа «НЫКЛОУ» текст начинался с бессмыслицы.

При выборе ключа «КЛОУНЫ» текст начинался осмысленно.

Был выбран ключ КЛОУНЫ.

Программа выполнила финальную расшифровку полного текста с восстановлением пробелов и знаков препинания:

НО ПРЕЖДЕ НЕЖЕЛИ МЫ СКАЖЕМ, КТО ТАКОВ БЫЛ ПОРУЧИК ПИРОГОВ, НЕ

МЕШАЕТ КОЕ-ЧТО РАССКАЗАТЬ О ТОМ ОБЩЕСТВЕ, К КОТОРОМУ ПРИНАДЛЕЖАЛ ПИРОГОВ. ЕСТЬ

ОФИЦЕРЫ, СОСТАВЛЯЮЩИЕ В ПЕТЕРБУРГЕ КАКОЙ-ТО СРЕДНИЙ КЛАСС ОБЩЕСТВА. НА ВЕЧЕРЕ,

НА ОБЕДЕ У СТАТСКОГО СОВЕТНИКА ИЛИ У ДЕЙСТВИТЕЛЬНОГО СТАТСКОГО, КОТОРЫЙ

ВЫСЛУЖИЛ ЭТОТ ЧИН СОРОКАЛЕТНИМИ ТРУДАМИ, ВЫ ВСЕГДА НАЙДЕТЕ ОДНОГО ИЗ НИХ.

НЕСКОЛЬКО БЛЕДНЫХ, СОВЕРШЕННО БЕСЦВЕТНЫХ, КАК ПЕТЕРБУРГ, ДОЧЕРЕЙ, ИЗ КОТОРЫХ ИНЫЕ

ПЕРЕЗРЕЛИ, ЧАЙНЫЙ СТОЛИК, ФОРТЕПИАНО, ДОМАШНИЕ ТАНЦЫ - ВСЕ ЭТО БЫВАЕТ НЕРАЗДЕЛЬНО

С СВЕТЛЫМ ЭПОЛЕТОМ, КОТОРЫЙ БЛЕЩЕТ ПРИ ЛАМПЕ, МЕЖДУ БЛАГОНРАВНОЙ БЛОНДИНКОЙ И

ЧЕРНЫМ ФРАКОМ БРАТЦА ИЛИ ДОМАШНЕГО ЗНАКОМОГО. ЭТИХ

ХЛАДНОКРОВНЫХ ДЕВИЦ ЧРЕЗВЫЧАЙНО ТРУДНО РАСШЕВЕЛИТЬ И ЗАСТАВИТЬ СМЕЯТЬСЯ; ДЛЯ

ЭТОГО НУЖНО БОЛЬШОЕ ИСКУССТВО ИЛИ, ЛУЧШЕ СКАЗАТЬ, СОВСЕМ НЕ ИМЕТЬ НИКАКОГО

ИСКУССТВА. НУЖНО ГОВОРИТЬ ТАК, ЧТОБЫ НЕ БЫЛО НИ СЛИШКОМ УМНО, НИ СЛИШКОМ СМЕШНО,

ЧТОБЫ ВО ВСЕМ БЫЛА ТА МЕЛОЧЬ, КОТОРУЮ ЛЮБЯТ ЖЕНЩИНЫ.

В ЭТОМ НАДОБНО ОТДАТЬ

СПРАВЕДЛИВОСТЬ ОЗНАЧЕННЫМ ГОСПОДАМ. ОНИ ИМЕЮТ ОСОБЕННЫЙ ДАР ЗАСТАВЛЯТЬ СМЕЯТЬСЯ

И СЛУШАТЬ ЭТИХ БЕСЦВЕТНЫХ КРАСАВИЦ. ВОСКЛИЦАНИЯ, ЗАДУШАЕМЫЕ СМЕХОМ: "АХ,

ПЕРЕСТАНЬТЕ! НЕ СТЫДНО ЛИ ВАМ ТАК СМЕШИТЬ!" - БЫВАЮТ ИМ ЧАСТО ЛУЧШЕЮ НАГРАДОЮ. В

ВЫСШЕМ КЛАССЕ ОНИ ПОПАДАЮТСЯ ОЧЕНЬ РЕДКО ИЛИ, ЛУЧШЕ СКАЗАТЬ,

НИКОГДА. ОТТУДА ОНИ СОВЕРШЕННО ВЫТЕСНЕНЫ ТЕМ, ЧТО НАЗЫВАЮТ В ЭТОМ ОБЩЕСТВЕ

АРИСТОКРАТАМИ; ВПРОЧЕМ, ОНИ СЧИТАЮТСЯ УЧЕНЫМИ И ВОСПИТАННЫМИ ЛЮДЬМИ. ОНИ ЛЮБЯТ

ПОТОЛКОВАТЬ ОБ ЛИТЕРАТУРЕ; ХВАЛЯТ БУЛГАРИНА, ПУШКИНА И ГРЕЧА И ГОВОРЯТ С

ПРЕЗРЕНИЕМ И ОСТРОУМНЫМИ КОЛКОСТЯМИ ОБ А. А. ОРЛОВЕ.

4. Листинг программы:

```
import tkinter as tk
from tkinter import ttk, messagebox
import re
from collections import defaultdict

ALPHABET = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
ALPHABET_SIZE = 32

# https://studfile.net/preview/1725830/page:36/
RU_FREQ = [
0.080,0.016,0.045,0.014,0.072,0.076,0.007,0.016,0.062,0.010,
0.035,0.028,0.026,0.065,0.090,0.023,0.014,0.054,0.063,0.053,
0.014,0.038,0.004,0.045,0.014,0.006,0.016,0.016,0.003,0.018,
0.018,0.020
]

current_keys=[]
clean_text=""
original_text=""



# ищет повторяющийся текст заданной длины
def find_distances(text,n):
```

```
    m=defaultdict(list)

    for i in range(len(text)-n+1):
        m[text[i:i+n]].append(i)

    res=[]

    for v in m.values():
        if len(v)>1:
            for i in range(len(v)-1):
                res.append(v[i+1]-v[i])

    return res

# Индекс совпадений
def calculate_ic(text):

    n=len(text)

    if n<=1:
        return 0

    f=[0]*ALPHABET_SIZE

    for c in text:
        idx=ALPHABET.find(c)
        if idx!=-1:
            f[idx]+=1

    s=0
    for x in f:
        s+=x*(x-1)

    return s/(n*(n-1))


def calculate_average_ic(text,key_len):

    total=0

    for i in range(key_len):

        col=""

        for j in range(i,len(text),key_len):
            col+=text[j]

        if len(col)>1:
            total+=calculate_ic(col)

    return total/key_len

# Поиск длины ключа
def find_key_length(text):

    distances=find_distances(text,3)
```

```
    best_mu=-1

    if distances:

        votes={}

        for length in range(2,11):

            count=0

            for d in distances:
                if d%length==0:
                    count+=1

            votes[length]=count

        top3=sorted(votes,key=votes.get,reverse=True)[:3]

        min_diff=999

        for length in top3:

            ic=calculate_average_ic(text,length)
            diff=abs(ic-0.055)

            if diff<min_diff and diff<0.025:
                min_diff=diff
                best_mu=length

    if best_mu==-1:

        max_ic=-1

        for length in range(1,11):

            ic=calculate_average_ic(text,length)

            if ic>max_ic:
                max_ic=ic
                best_mu=length

    return best_mu if best_mu!=-1 else 1

# Частотный анализ
def get_top_shifts(col,n):

    counts=[0]*ALPHABET_SIZE
    total=0

    for c in col:

        idx=ALPHABET.find(c)

        if idx!=-1:
            counts[idx]+=1
            total+=1
```

```
    if total==0:
        return ['A']

    scores=[]

    for shift in range(ALPHABET_SIZE):

        err=0

        for i in range(ALPHABET_SIZE):

            orig=(i-shift+ALPHABET_SIZE)%ALPHABET_SIZE
            obs=counts[i]/total

            err+=(obs-RU_FREQ[orig])**2 # Метод наименьших квадратов

        scores.append((shift,err))

    scores.sort(key=lambda x:x[1])

    res=[]

    for i in range(min(n,len(scores))):
        res.append(ALPHABET[scores[i][0]])

    return res

# Расшифровка Виженера

def decrypt_preserve(original,key):

    res=[]
    key_index=0

    for c in original:

        # переводим весь текст в верхний регистр
        upper=c.upper()

        # ё = Е
        if upper=="Ё":
            upper="Е"

        cIdx=ALPHABET.find(upper)

        if cIdx!=-1:

            kIdx=ALPHABET.find(key[key_index%len(key)])

            dec=(cIdx-kIdx)%ALPHABET_SIZE
            ch=ALPHABET[dec]

            if c.islower():
                res.append(ch.lower())
            else:
                res.append(ch)
```

```
                key_index+=1

            else:
                res.append(c)

        return "".join(res)


    # ---------------- GUI ----------------

    class App:

        def __init__(self,root):

            self.root=root
            root.title("Криптоанализ Виженера")

            frame1=tk.LabelFrame(root,text="Введите текст")
            frame1.pack(fill="both",expand=True,padx=5,pady=5)

            self.input_text=tk.Text(frame1,height=8)
            self.input_text.pack(fill="both",expand=True)

            btn_frame=tk.Frame(root)
            btn_frame.pack(pady=5)

            tk.Button(btn_frame,text="Анализировать",
                      command=self.analyze).grid(row=0,column=0,padx=5)

            tk.Button(btn_frame,text="Расшифровать",
                      command=self.decrypt).grid(row=0,column=1,padx=5)

            tk.Button(btn_frame,text="Очистить поля",

command=self.clear_fields).grid(row=0,column=2,padx=5)

            frame2=tk.LabelFrame(root,text="Выберите ключ")
            frame2.pack(fill="x",padx=5,pady=5)

            self.combo=ttk.Combobox(frame2,state="readonly")
            self.combo.pack(fill="x",padx=5,pady=5)

            frame3=tk.LabelFrame(root,text="Расшифрованный текст")
            frame3.pack(fill="both",expand=True,padx=5,pady=5)

            self.result=tk.Text(frame3)
            self.result.pack(fill="both",expand=True)


        def analyze(self):

            global clean_text,original_text,current_keys

            original_text=self.input_text.get("1.0",tk.END)

            if not original_text.strip():
                messagebox.showwarning("Ошибка","Введите текст")
```

```
            return

        # убираем всякую всячину
        clean_text=original_text.upper()
        clean_text=clean_text.replace("Ё","Е")
        clean_text=re.sub("[^А-Я]","",clean_text)

        if len(clean_text)<20:
            messagebox.showerror("Ошибка","Минимум 20 букв")
            return

        # ищем длину ключа
        mu=find_key_length(clean_text)

        base_key=""

        # находим ключи
        for i in range(mu):

            col=""

            for j in range(i,len(clean_text),mu):
                col+=clean_text[j]

            base_key+=get_top_shifts(col,1)[0]

        current_keys=[]

        for i in range(mu):

            shifted=base_key[i:]+base_key[:i]
            current_keys.append(shifted)

        self.combo["values"]=current_keys

        if current_keys:
            self.combo.current(0)


    def decrypt(self):

        key=self.combo.get()

        if not key:
            messagebox.showwarning("Ошибка","Выберите ключ")
            return

        # переводим текст
        text=decrypt_preserve(original_text,key)

        self.result.delete("1.0",tk.END)
        self.result.insert(tk.END,text)


    def clear_fields(self):

        global current_keys,clean_text,original_text
```

```
        self.input_text.delete("1.0",tk.END)
        self.result.delete("1.0",tk.END)

        self.combo.set("")
        self.combo["values"]=[]

        current_keys=[]
        clean_text=""
        original_text=""

root=tk.Tk()
app=App(root)
root.mainloop()
```

5. Результат работы программы

На рисунке 1 представлен результат работы программы.

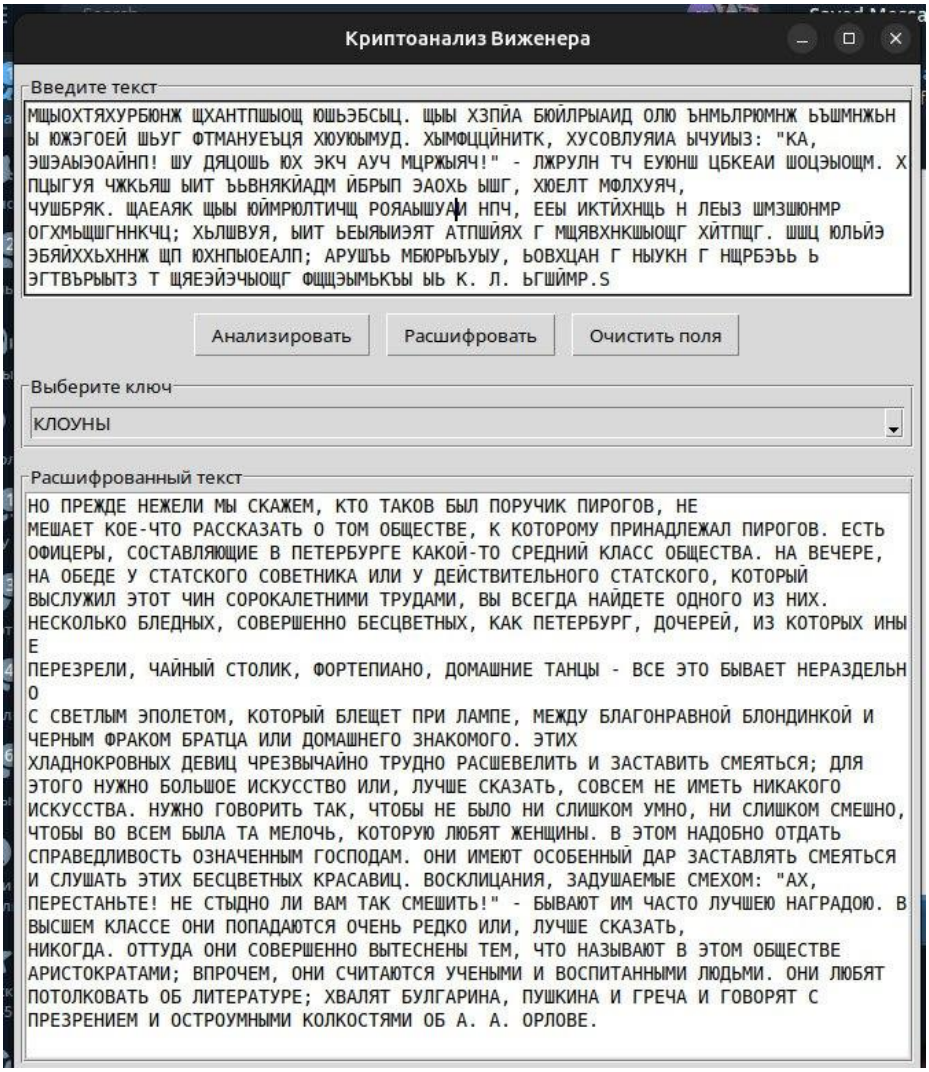

Рисунок 1 – Результат работы программы

6. **Заключение**

В ходе выполнения практической работы были изучены принципы полиалфавитных шифров и освоены методы статистического криптоанализа, включая тест Касиски и индекс совпадений (IC). На языкеPython с использованием библиотеки tkinter разработано приложение, реализующее автоматический поиск длины ключа, частотный анализ столбцов и генерацию циклических сдвигов для устранения неоднозначности. В результате успешно взломан шифр Виженера: определен секретный ключ и полностью расшифровано литературное произведение с сохранением исходного форматирования текста.